

MEDALLION ARCHITECTURE · ETL

Loading the Silver Layer

How `silver.load_silver` cleanses, standardizes, and reshapes six Bronze tables from the CRM and ERP systems into trusted Silver tables.



`silver.load_silver` · T-SQL Stored Procedure



Pipeline Overview

Every table follows the same two-step pattern inside one TRY / CATCH block

1 TRUNCATE

Each Silver table is fully truncated before reload, so the layer always reflects the latest Bronze snapshot.

2 INSERT · TRANSFORM

Cleansed, standardized data is inserted — trimming text, mapping codes, fixing dates, and deduplicating rows.

3 LOG DURATION

Start/end timestamps are captured per table and printed, plus one total duration for the whole batch.

Six Bronze tables are loaded, in this order:

crm_cust_info

Customer master data

crm_prd_info

Product catalog

crm_sales_details

Sales transactions

erp_cust_az12

ERP customer demographics

erp_loc_a101

ERP customer location

erp_px_cat_g1v2

ERP product categories

SRC bronze.crm_cust_info

TGT silver.crm_cust_info

Key Transformations

1 Deduplicate customers

ROW_NUMBER() partitioned by cst_id, ordered by cst_create_date DESC.

2 Drop incomplete rows

Rows with a NULL cst_id are filtered out before deduplication.

3 Trim text fields

cst_firstname and cst_lastname are stripped of whitespace.

4 Standardize codes

Marital status (S/M) and gender (F/M) mapped to readable values; else 'n/a'.

```
T-SQL - from silver.load_silver

SELECT
  cst_id,
  cst_key,
  TRIM(cst_firstname) AS cst_firstname,
  TRIM(cst_lastname) AS cst_lastname,
  CASE
    WHEN UPPER(TRIM(cst_marital_status)) = 'S'
      THEN 'Single'
    WHEN UPPER(TRIM(cst_marital_status)) = 'M'
      THEN 'Married'
    ELSE 'n/a'
  END AS cst_marital_status,
  CASE
    WHEN UPPER(TRIM(cst_gndr)) = 'F' THEN 'Female'
    WHEN UPPER(TRIM(cst_gndr)) = 'M' THEN 'Male'
    ELSE 'n/a'
  END AS cst_gndr,
  cst_create_date
FROM (
  SELECT *,
    ROW_NUMBER() OVER (
      PARTITION BY cst_id
      ORDER BY cst_create_date DESC
    ) AS flag_last
  FROM bronze.crm_cust_info
  WHERE cst_id IS NOT NULL
) t
WHERE flag_last = 1;
```

SRC bronze.crm_prd_info

TGT silver.crm_prd_info

Key Transformations

1 Split the product key

First 5 chars (with '-' → '_') become cat_id; remainder is the cleaned prd_key.

2 Default missing cost

NULL prd_cost values are replaced with 0 via ISNULL().

3 Map product line codes

M/R/S/T expand to Mountain/Road/Other Sales/Touring; else 'n/a'.

4 Derive the end date

LEAD() looks at the next row's start date (per prd_key) minus 1 day.

```
T-SQL - from silver.load_silver

SELECT
    prd_id,
    REPLACE(SUBSTRING(prd_key,1,5), '-', '_')
        AS cat_id,
    SUBSTRING(prd_key,7,LEN(prd_key))
        AS prd_key,
    prd_nm,
    ISNULL(prd_cost, 0) AS prd_cost,
    CASE
        WHEN UPPER(TRIM(prd_line)) = 'M'
            THEN 'Mountain'
        WHEN UPPER(TRIM(prd_line)) = 'R' THEN 'Road'
        WHEN UPPER(TRIM(prd_line)) = 'S'
            THEN 'Other Sales'
        WHEN UPPER(TRIM(prd_line)) = 'T'
            THEN 'Touring'
        ELSE 'n/a'
    END AS prd_line,
    CAST(prd_start_dt AS DATE) AS prd_start_dt,
    CAST(
        LEAD(prd_start_dt) OVER (
            PARTITION BY prd_key
            ORDER BY prd_start_dt
        ) - 1 AS DATE
    ) AS prd_end_dt
FROM bronze.crm_prd_info;
```

SRC bronze.crm_sales_details

TGT silver.crm_sales_details

Key Transformations

1 Validate integer dates

8-digit int dates; 0 or wrong length → NULL, else CAST to DATE.

2 Recalculate sales

If missing, ≤ 0 , or $\neq \text{qty} \times |\text{price}|$ → recomputed as $\text{qty} \times \text{ABS}(\text{price})$.

3 Derive unit price

If missing or ≤ 0 → sales / quantity, guarded with NULLIF.

```
T-SQL - from silver.load_silver

SELECT
  sls_ord_num, sls_prd_key, sls_cust_id,
  CASE WHEN sls_order_dt = 0
        OR LEN(sls_order_dt) != 8 THEN NULL
        ELSE CAST(CAST(sls_order_dt AS VARCHAR)
                  AS DATE)
  END AS sls_order_dt,
  CASE WHEN sls_ship_dt = 0
        OR LEN(sls_ship_dt) != 8 THEN NULL
        ELSE CAST(CAST(sls_ship_dt AS VARCHAR)
                  AS DATE)
  END AS sls_ship_dt,
  CASE WHEN sls_due_dt = 0
        OR LEN(sls_due_dt) != 8 THEN NULL
        ELSE CAST(CAST(sls_due_dt AS VARCHAR)
                  AS DATE)
  END AS sls_due_dt,
  CASE WHEN sls_sales IS NULL OR sls_sales <= 0
        OR sls_sales != sls_quantity*ABS(sls_price)
        THEN sls_quantity * ABS(sls_price)
        ELSE sls_sales
  END AS sls_sales,
  sls_quantity,
  CASE WHEN sls_price IS NULL OR sls_price <= 0
        THEN sls_sales / NULLIF(sls_quantity, 0)
        ELSE sls_price
  END AS sls_price
FROM bronze.crm_sales_details;
```

SRC bronze.erp_cust_az12

TGT silver.erp_cust_az12

Key Transformations

1 Strip 'NAS' prefix

Where cid begins with 'NAS', the prefix is removed.

2 Nullify future birthdates

Any bdate later than today is treated as bad data → NULL.

3 Standardize gender

F/FEMALE → Female, M/MALE → Male; else 'n/a'.

```
T-SQL - from silver.load_silver

SELECT
  CASE
    WHEN cid LIKE 'NAS%'
      THEN SUBSTRING(cid, 4, LEN(cid))
    ELSE cid
  END AS cid,
  CASE
    WHEN bdate > GETDATE() THEN NULL
    ELSE bdate
  END AS bdate,
  CASE
    WHEN UPPER(TRIM(gen)) IN ('F','FEMALE')
      THEN 'Female'
    WHEN UPPER(TRIM(gen)) IN ('M','MALE')
      THEN 'Male'
    ELSE 'n/a'
  END AS gen
FROM bronze.erp_cust_az12;
```



SRC bronze.erp_loc_a101

TGT silver.erp_loc_a101

Key Transformations

1 Clean the customer key

Hyphens are removed from cid to match the Silver key format.

2 Standardize country names

DE → Germany, US/USA → United States, blank/NULL → 'n/a'.

```
T-SQL - from silver.load_silver

SELECT
  REPLACE(cid, '-', '') AS cid,
  CASE
    WHEN TRIM(cntry) = 'DE' THEN 'Germany'
    WHEN TRIM(cntry) IN ('US', 'USA')
      THEN 'United States'
    WHEN TRIM(cntry) = '' OR cntry IS NULL
      THEN 'n/a'
    ELSE TRIM(cntry)
  END AS cntry
FROM bronze.erp_loc_a101;
```

SRC bronze.erp_px_cat_glv2

TGT silver.erp_px_cat_glv2

Key Transformations



Direct pass-through

id, cat, subcat, and maintenance are already clean — no CASE logic.



Why include it at all?

Truncate-and-reload still applies, keeping it in sync with Bronze.

 T-SQL — from silver.load_silver

```
SELECT
  id,
  cat,
  subcat,
  maintenance
FROM bronze.erp_px_cat_glv2;
```


Error Handling & Logging

The entire load runs inside one TRY / CATCH block, with timing captured at every step



BEGIN TRY

- Records @batch_start_time at the very start of the run
- Loads all six tables in sequence: CRM tables first, then ERP tables
- Each table's TRUNCATE + INSERT is timed with @start_time / @end_time
- PRINT statements report each table's load duration in seconds
- @batch_end_time is captured once all tables finish loading
- A final PRINT reports total batch duration for the whole procedure



BEGIN CATCH

If any step fails, execution jumps here and the procedure prints three diagnostics instead of throwing an unhandled error:

ERROR_MESSAGE () the text of what went wrong

ERROR_NUMBER () the SQL Server error code

ERROR_STATE () the error state, for diagnostics

Note: the CATCH block logs the error but does not re-throw or roll back — downstream steps should check for it explicitly.

Result

A trusted Silver Layer

Six Bronze tables — cleansed, standardized, and deduplicated — ready for the Gold layer's business-ready views.

6

tables loaded

2

source systems — CRM & ERP

1

TRY/CATCH wrapped procedure

```
EXEC silver.load_silver;
```

